

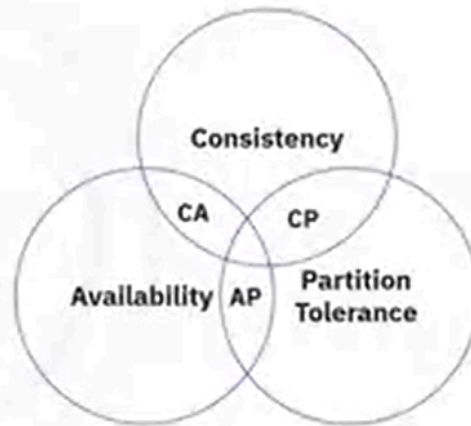
CAP Theorem

Consistency versus availability

- Early 2000s: Emerging of Hadoop, the first open big data architecture that allowed distributed storage and processing of large amounts of data
 - Services emerging in 2000s required distributed databases
 - Active and accessible worldwide
 - Always available
 - Relational databases: ACID-based, relied on data consistency
 - Availability and consistency seemed impossible
-
- In the early 2000s big data Hadoop architecture was being created as the first open source, distributed architecture that can store and process high volumes of data. At this time, more and more services were developed that required databases to be distributed as well. Actually, these businesses required not only that their services were active and accessible in most parts of the world, but also that their services were always available. For relational databases that rely so much on the consistency of data, the new concept of availability while having a distributed system seemed impossible, and this was proven by the CAP Theorem.

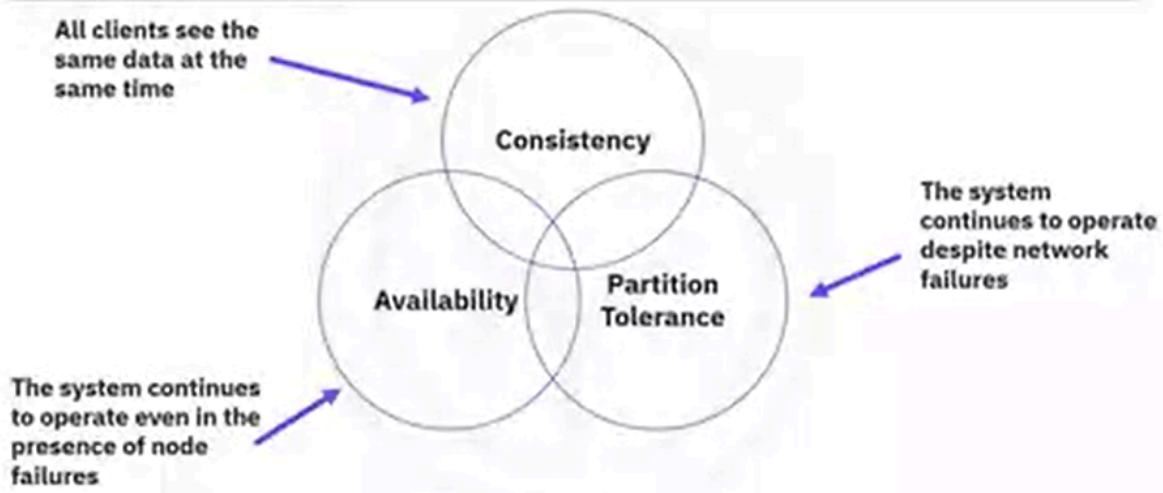
CAP theorem definition and history

- CAP Theorem (Brewer's theorem)
- Evolved by MIT professors Seth Gilbert and Nancy Lynch
- Consistency, Availability and Partition Tolerance (CAP)
- Only two of the three characteristics can be guaranteed in distributed systems



- The CAP Theorem is also called Brewer's Theorem, because it was first advanced by Professor Eric A. Brewer during a talk he gave on distributed computing in the year 2000. Two years later it was revised by MIT professors Seth Gilbert and Nancy Lynch, and there have been many other contributors since. The theorem states that there are three essential system requirements necessary for the successful design, implementation, and deployment of applications in distributed systems. These are Consistency, Availability, and Partition Tolerance, or CAP. A distributed system can guarantee delivery of only two of these three desired characteristics.

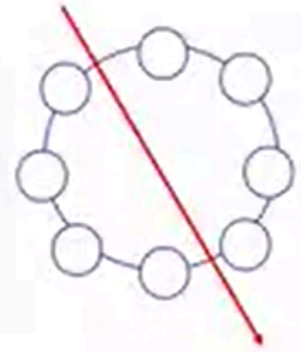
CAP theorem



- Now let's see what these three characteristics mean. Consistency refers to whether a system operates fully or not. Do all nodes within a cluster see all the data they are supposed to? Availability means just as it sounds. Does each request get a response outside of failure or success? Partition Tolerance represents the fact that a given system continues to operate even under circumstances of data loss or network failure. While consistency and availability seem straightforward, let's see what partition tolerance refers to.

Partition tolerance

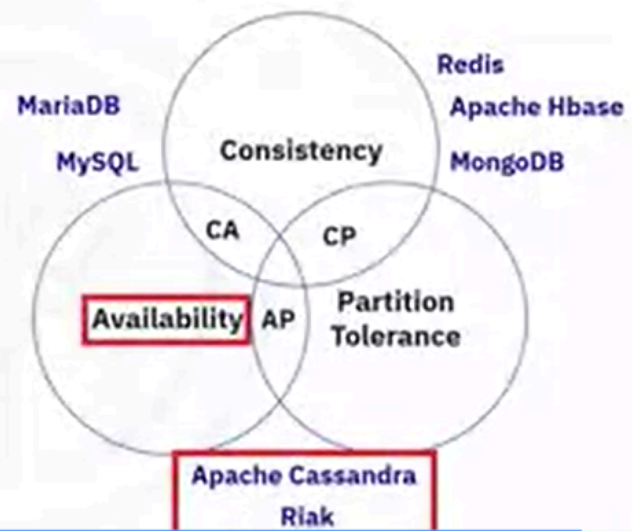
- A Partition is lost or temporarily delayed connection between nodes
 - Partition tolerance means that the cluster must work despite network issues
 - Distributed systems cannot avoid partitions and must be partition-tolerant
- Partition tolerance is a basic NoSQL feature



- A partition is a communications break within a distributed system—a lost or temporarily delayed connection between nodes. Partition tolerance means that the cluster must continue to work despite any number of communication breakdowns between nodes in the system. In distributed systems, partitions can't be avoided. Therefore, partition tolerance becomes a basic feature of native distributed systems such as NoSQL. In a cluster with eight distributed nodes, a network partition could occur, and communication will be broken between all the nodes.
- In our case, instead of one 8-node cluster we will have two smaller 4-node clusters available. Consistency between the two clusters will be achieved when network communication is re-established. Partition tolerance has become more of a necessity than an option in distributed systems. It is made possible by sufficiently replicating records across combinations of nodes and networks.

NoSQL: CP or AP

- NoSQL: A choice between consistency and availability
- MongoDB: Chooses consistency
- Apache Cassandra: Chooses availability
- These are tunable systems



(spacebar)

- For such systems as NoSQL, since partition tolerance is mandatory, a system can be either Consistent and Partition Tolerant (CP) or Available and Partition Tolerant (AP). Existing NoSQL systems, like MongoDB or Cassandra, can be classified using CAP Theorem. For example, MongoDB chooses consistency as the primary design driver of the solution, and Apache Cassandra chooses availability. This doesn't mean that MongoDB cannot be available, or that Cassandra cannot become fully consistent. It means that these solutions first ensure that they are consistent (in the case of MongoDB) or available (in the case of Cassandra) and the rest is tunable.

Understanding the CAP Theorem: A Simple Explanation

The CAP Theorem is a concept that helps us understand how distributed systems, like databases, work. It states that in any distributed system, you can only guarantee two out of three important features: Consistency, Availability, and Partition Tolerance. Think of it like a pizza: you can choose two toppings, but you can't have all three at once!

- **Consistency** means that every part of the system has the same data at the same time. Imagine if you and your friend are looking at the same menu, and you both see the same options.
- **Availability** means that the system is always ready to respond to requests, like a restaurant that is always open for customers.
- **Partition Tolerance** means that the system can still work even if some parts can't communicate with each other, like a group of friends who can still chat even if some of them are in different rooms.

In summary, the CAP Theorem helps us understand the trade-offs we have to make when designing systems that need to be reliable and efficient.

How does CAP Theorem apply to NoSQL databases?

The CAP Theorem is particularly relevant to NoSQL databases because these databases are often designed to handle large amounts of data across distributed systems. Here's how it applies:

1. **Partition Tolerance:** NoSQL databases are built to be partition-tolerant, meaning they can continue to operate even if some nodes in the system cannot communicate with each other. This is crucial for maintaining service availability in real-world scenarios where network failures can occur.
2. **Consistency vs. Availability:** NoSQL databases must choose between consistency and availability:
 - **Consistency (CP):** Some NoSQL databases, like MongoDB, prioritize consistency. This means that when data is written, all nodes will reflect that change before any read operations can occur. This ensures that users always see the most up-to-date data.
 - **Availability (AP):** Other NoSQL databases, like Apache Cassandra, prioritize availability. This means that the system will always respond to

requests, even if it means that some nodes might return outdated data. This is useful for applications that require high uptime.

In summary, NoSQL databases leverage the CAP Theorem to balance the trade-offs between consistency, availability, and partition tolerance based on the specific needs of the application they support.

What would happen if a NoSQL database prioritized both consistency and availability?

If a NoSQL database attempted to prioritize both **consistency** and **availability**, it would face significant challenges, especially in a distributed system. Here's what could happen:

1. **Increased Latency:** To ensure consistency, the database would need to wait for all nodes to confirm that they have the same data before responding to any requests. This could lead to increased response times, making the system less available to users.
2. **Potential for Downtime:** If a network partition occurs (where some nodes cannot communicate with others), the system might have to choose between becoming unavailable or returning inconsistent data. If it prioritizes consistency, it may refuse to respond to requests until all nodes are synchronized, leading to downtime.
3. **Complexity in Design:** Designing a system that guarantees both consistency and availability in a distributed environment is complex. It may require sophisticated algorithms and mechanisms, which can increase the risk of bugs and performance issues.
4. **Trade-offs:** In practice, achieving both consistency and availability often means sacrificing partition tolerance, which is not ideal in real-world scenarios where network failures are common. This could lead to a system that is not resilient and fails to meet user expectations.

In summary, while it may seem desirable to have both consistency and availability, the CAP Theorem indicates that in a distributed system, you cannot have all three

characteristics simultaneously. Most NoSQL databases choose to prioritize either consistency or availability to ensure better performance and reliability.

How can NoSQL databases balance consistency and availability?

NoSQL databases balance **consistency** and **availability** through various strategies and design choices. Here are some common approaches:

1. Eventual Consistency:

- Many NoSQL databases adopt an eventual consistency model, where updates to data are propagated to all nodes over time. This means that while data may not be immediately consistent across all nodes, it will eventually become consistent. This approach allows for high availability since the system can respond to requests even during updates.

2. Tunable Consistency:

- Some NoSQL databases, like Cassandra, offer tunable consistency levels. This allows developers to choose the level of consistency required for specific operations. For example, they can opt for strong consistency for critical transactions while allowing eventual consistency for less critical ones. This flexibility helps balance the trade-offs based on application needs.

3. Quorum Reads and Writes:

- A quorum-based approach requires a majority of nodes to agree on a read or write operation before it is considered successful. This ensures that the data is consistent while still allowing for high availability. For example, if a system has five nodes, a quorum might require at least three nodes to respond.

4. Replication:

- NoSQL databases often use data replication across multiple nodes. By replicating data, the system can ensure that even if one node fails, others

can still provide access to the data. This helps maintain availability while also allowing for consistency checks across replicas.

5. Conflict Resolution:

- In systems that allow concurrent updates, conflict resolution strategies are employed to handle discrepancies. Techniques like last-write-wins, versioning, or custom conflict resolution logic can help maintain consistency while ensuring that the system remains available.

By implementing these strategies, NoSQL databases can effectively balance consistency and availability, allowing them to meet the demands of modern applications.

What would happen if a NoSQL database prioritized consistency over availability?

If a NoSQL database prioritized **consistency** over **availability**, several outcomes could occur:

1. **Increased Downtime:** The system may become unavailable during network partitions or when nodes cannot communicate. If the database cannot ensure that all nodes have the same data, it may refuse to process requests until consistency is achieved, leading to potential downtime for users.
2. **Slower Response Times:** To maintain consistency, the database would need to wait for all relevant nodes to confirm data updates before responding to any requests. This could result in slower response times, as users may experience delays while the system ensures that all nodes are synchronized.
3. **User Experience Impact:** Users may face interruptions or delays when trying to access data, especially during high-traffic periods or when network issues arise. This could lead to frustration and a negative experience for users relying on the system for real-time data access.
4. **Complexity in Handling Failures:** The system would need to implement complex mechanisms to handle failures and ensure that data remains

consistent. This could increase the risk of bugs and make the system harder to maintain.

5. **Trade-offs in Application Design:** Developers may need to design applications with the understanding that certain operations may not be available at all times. This could require additional logic in the application layer to handle scenarios where data is temporarily inaccessible.

In summary, while prioritizing consistency can ensure that users always see the most accurate data, it can lead to significant trade-offs in terms of availability and performance. This approach may be suitable for applications where data accuracy is critical, but it can also create challenges in maintaining a responsive and reliable system.